

CSE276 · ARTIFICIAL INTELLIGENCE FOUNDATIONS

UNIT 2

Problem Solving using Search, Knowledge Representation and Reasoning

How a machine finds a path it was never shown - and how it stores what it knows so it can reason about it.

BFS · DFS · Best First · Hill Climbing · A* · Heuristics · Semantic Networks · Frames · Production & Expert Systems · Propositional & Predicate Logic · Bayes' Theorem

UNIT II ROADMAP

What This Unit Covers - Two Halves

UNIT 2

Part A searches for an answer. Part B stores what you already know so you can deduce one.

- **PART A - Problem Solving using Search**
 - Uninformed search: Breadth First and Depth First
 - Informed search: Best First, Hill Climbing and A*
 - Heuristic functions - what makes one admissible
 - Comparison and applications of the search algorithms
- **PART B - Knowledge Representation and Reasoning**
 - Knowledge representation, semantic networks and frames
 - Production systems and expert systems
 - Propositional logic and first-order predicate logic
 - Reasoning under uncertainty with Bayes' theorem

Uninformed

BFS, DFS

Informed

Best First, Hill Climbing, A*

Representation

networks, frames, rules

Reasoning

logic and Bayes

01

SECTION

Uninformed Search

IN THIS SECTION

- What 'uninformed' actually means
- Breadth First Search
- Depth First Search
- Completeness, optimality, time and space

2.1 UNINFORMED SEARCH

Uninformed Search - No Knowledge of Where the Goal Is

UNIT 2

KEY CONCEPT · Also called blind search

The defining property the algorithm knows the states and the legal actions, but has NO information about which direction the goal lies in.

What it can use the problem definition only - initial state, actions, transition model, goal test, path cost.

What it cannot use any estimate of the remaining distance. That estimate is a heuristic, and it makes a search INFORMED.

The consequence it must explore systematically, which becomes expensive very quickly.

The two you must know Breadth First Search explores level by level; Depth First Search follows one branch to its end.

Both consume the same input the five-component problem definition from Unit I. That is why formulation came first.

2.1 BFS VS DFS

Breadth First Search vs Depth First Search

UNIT 2

One line of code separates them: whether you take from the front of the frontier or the back.

	Breadth First Search	Depth First Search
Frontier structure	Queue - first in, first out	Stack - last in, first out
Explores	All nodes at depth d before any at depth $d+1$	One branch as deep as possible, then backtracks
Complete?	Yes, when the branching factor is finite	No - it can descend an infinite branch forever
Optimal?	Yes, but ONLY when every step costs the same	No
Time complexity	$O(b^d)$	$O(b^m)$
Space complexity	$O(b^d)$ - stores the entire frontier	$O(b \cdot m)$ - stores one branch only
Use it when	The shallowest solution is wanted and memory allows	Memory is tight and any solution will do

REASON IT OUT Why does DFS use so much less memory?

2.1 UNINFORMED SEARCH

UNIT 2

BFS Is Optimal - But Only Under One Condition

COMMON PITFALL · The condition students forget, and lose marks for

The claim you will read 'BFS is optimal.'

The condition attached BFS is optimal only when every step costs the SAME.

Why BFS finds the solution with the fewest EDGES. It has no notion of cost, so it treats every edge as costing 1.

Where it breaks on a road network, the route with the fewest junctions is frequently not the shortest in kilometres.

The fix Uniform Cost Search - expand the frontier node with the lowest path cost $g(n)$ rather than the shallowest one.

Write it precisely 'BFS is optimal when all step costs are equal.' The clause is the answer.

The exam-safe summary

- BFS: complete, optimal if step costs are equal, $O(b^d)$ time AND space
- DFS: incomplete in infinite spaces, not optimal, $O(b \cdot m)$ space
- Space is usually what kills you first, not time

02

SECTION

Informed Search and Heuristics

IN THIS SECTION

- What a heuristic function is
- Best First Search
- Hill Climbing and local optima
- A* Search, and what makes it optimal

2.2 HEURISTIC FUNCTIONS

The Three Functions of Informed Search

UNIT 2

Uninformed search uses only g . Best First uses only h . A^ uses both.*

G(N) - THE COST ALREADY PAID

$g(n)$ = the actual path cost from the start to node n

Known exactly, because you travelled it.

H(N) - THE HEURISTIC ESTIMATE

$h(n)$ = an ESTIMATE of the cost from n to the goal

A guess supplied by you, using knowledge of the problem. Straight-line distance is the classic example.

F(N) - THE EVALUATION FUNCTION

$f(n) = g(n) + h(n)$

Estimated total cost of a path through n . A^* expands the node with the smallest f .

ADMISSIBILITY - THE CONDITION FOR A^* TO BE OPTIMAL

$h(n) \leq$ true cost from n to the goal, for every n

The heuristic must NEVER overestimate. An optimistic heuristic keeps A^* optimal; an overestimating one can make it miss the best path.

WHICH ALGORITHM USES WHAT

Uniform Cost $f(n) = g(n)$ - cost so far only

Best First (greedy) $f(n) = h(n)$ - estimate only; fast but not optimal

A^* $f(n) = g(n) + h(n)$ - both; optimal when h is admissible

Hill Climbing moves only to a better h ; keeps no frontier at all

$h(n) = 0$ reduces A^* to Uniform Cost Search

2.2 INFORMED SEARCH

Three Informed Search Strategies

UNIT 2

All three use a heuristic. They differ in what else they consider.

STRATEGY 1

Best First (greedy)

- Expands whichever node LOOKS closest: $f = h$
- Fast, and often finds a solution quickly
- Not optimal - it ignores the cost already paid
- Can be led badly astray by a misleading heuristic

STRATEGY 2

Hill Climbing

- Moves only to a neighbour with a better h
- Keeps NO frontier - constant memory
- Stops at a local optimum, plateau or ridge
- Fixes: random restarts, simulated annealing

STRATEGY 3

A* Search

- Expands the node with the smallest $f = g + h$
- Complete AND optimal when h is admissible
- Optimally efficient - no algorithm expands fewer nodes
- Cost: it stores the frontier, so memory is the limit

THE TRADE-OFF

Guarantees against memory

- A* pays memory for a guarantee
- Hill Climbing pays the guarantee for memory
- Best First sits between the two
- Neither is free

2.2 WORKED COMPARISON

UNIT 2

Comparing Search on the 8-Puzzle

THE PROBLEM

- A 3x3 sliding puzzle, a genuine 18 moves from the goal
- Two admissible heuristics available:
- h_1 = number of misplaced tiles
- h_2 = total Manhattan distance of every tile from its home
- These are the figures you will measure yourself in Practical 2

WHY IT MATTERS

- A better admissible heuristic explores strictly fewer nodes
- $h(n) = 0$ makes A* degenerate into Uniform Cost Search
- The 15-puzzle has $\sim 10^{13}$ states - heuristics are what make it solvable at all

STEP-BY-STEP SOLUTION

BFS - no heuristic 13,001 states explored, and it does find the optimal 18 moves

Best First - h_2 alone, no g only 72 states explored, but it returns 26 moves. Fastest, and the only one that is wrong

A* with h_1 (misplaced tiles) 1,610 states - h_1 gives real guidance, but it is a weak estimate

A* with h_2 (Manhattan) 444 states, still optimal. Roughly a 30-fold saving over BFS

BFS and both A* variants agree on 18 moves BFS is optimal here because every step costs 1; A* is optimal because both heuristics are admissible

Why h_2 dominates h_1 $h_2(n) \geq h_1(n)$ for every n , while both stay admissible. A heuristic closer to the truth prunes more.

ANSWER Same optimal answer from three of the four. The fourth was fastest and wrong - that is what dropping g buys you.

2.2 HILL CLIMBING

UNIT 2

Hill Climbing and the Local Optimum

COMMON PITFALL · Cheap memory, no guarantee whatsoever

How it works look at the neighbours, move to the best one, repeat. Stop when no neighbour is better.

Local optimum every neighbour is worse, but you are nowhere near the goal. The algorithm stops anyway.

Plateau a flat region where all neighbours are equal, so there is no signal about which way to go.

Ridge a narrow rising path where every single-step move goes down, even though progress is possible diagonally.

Fix 1 - random restarts when stuck, restart from a new random state and keep the best result. Cheap, and surprisingly effective.

Fix 2 - simulated annealing sometimes accept a WORSE move, with a probability that decreases over time, so the search can escape a local optimum early on.

DISCUSS Why does accepting a worse move ever help?

2.3 COMPARISON AND APPLICATIONS

UNIT 2

Comparison and Applications of the Search Algorithms

* if the branching factor is finite ** if step costs are equal *** if the heuristic is admissible

Algorithm	Complete	Optimal	Space	Typical application
Breadth First	Yes*	Yes**	$O(b^d)$	Shortest hop count; small state spaces
Depth First	No	No	$O(b \cdot m)$	Memory-limited search; puzzle solving
Uniform Cost	Yes*	Yes	$O(b^d)$	Weighted routing where costs differ
Best First	No	No	$O(b^d)$	Fast approximate answers
Hill Climbing	No	No	$O(1)$	Optimisation, scheduling, 8-queens
A*	Yes*	Yes***	$O(b^d)$	Route planning, robot path planning, games

CHECK YOUR UNDERSTANDING

Quick Check - Search

UNIT 2

QUICK CHECK

A* uses a heuristic that sometimes **OVERESTIMATES** the remaining cost. What is the consequence?

- A** A* becomes incomplete and may never terminate
- B** A* still terminates but may return a sub-optimal path
- C** A* becomes faster with no downside at all
- D** A* degenerates into Breadth First Search

ANSWER: B Admissibility - never overestimating - is the condition for A*'s optimality. An overestimating heuristic can make A* discard the truly best path because it looks too expensive, so it still finds a solution, just not necessarily the cheapest one.

03

SECTION

Knowledge Representation

IN THIS SECTION

- What knowledge representation is for
- Semantic networks
- Frames
- Choosing a representation

2.4 KNOWLEDGE REPRESENTATION

Why Represent Knowledge at All?

UNIT 2

KEY CONCEPT · The other half of AI - and the answer to the first winter

Search finds a path but it must explore. On a hard problem it explores an impossible number of states.

Knowledge replaces search if you already know that a bishop stays on one colour, you never search moves that break the rule.

The historical turn after the first AI winter, the field's answer was to stop searching harder and start encoding what experts know.

The requirements a representation must be expressive enough to say what you mean, and structured enough for a machine to reason over.

The four you must know semantic networks, frames, production systems, and logic.

They are not rivals real systems combine them, and each makes different things easy.

2.4 SEMANTIC NETWORKS AND FRAMES

Semantic Networks and Frames

UNIT 2

Both give you inheritance. One is built from edges, the other from records.

Semantic networks

Structure a graph. Nodes are concepts; labelled edges are relationships.

Key relations is-a (class membership) and has-a (properties).

Inheritance a property stated at a general node is inherited by everything below it - state it once.

Good for taxonomies, class hierarchies, anything where relationships dominate.

Weak for procedures, exceptions and quantified statements.

Frames

Structure a record of SLOTS and their values, one frame per concept.

Defaults a slot can carry a default value that a more specific frame overrides.

Inheritance frames link to parent frames, so unfilled slots fall back.

Good for structured objects with typical values - this is where object-oriented programming came from.

Weak for arbitrary relations between objects, and for reasoning that is not slot lookup.

2.4 PRODUCTION AND EXPERT SYSTEMS

UNIT 2

Production Systems and Expert Systems

KEY CONCEPT · IF-THEN rules plus an engine that fires them

A production rule IF <conditions> THEN <action or conclusion>. The conditions are matched against known facts.

The three components a KNOWLEDGE BASE of rules, a WORKING MEMORY of current facts, and an INFERENCE ENGINE that decides which rule fires.

Forward chaining start from the facts and fire every rule whose conditions hold, adding conclusions until nothing new is derived. Data-driven.

Backward chaining start from a hypothesis and work backwards, asking what would have to be true. Goal-driven, and the natural choice for diagnosis.

The explanation facility because reasoning is a chain of named rules, the system can show WHY it concluded something - which is why expert systems survive in medicine and law.

The limits brittleness outside the rule set, and the knowledge acquisition bottleneck - every rule must be extracted from a human by hand.

DECIDE Diagnosing a fault from symptoms: forward or backward chaining?

04

SECTION

Logic and Reasoning

IN THIS SECTION

- Propositional logic
- First-order predicate logic
- What quantifiers buy you
- Reasoning under uncertainty with Bayes' theorem

2.5 LOGIC

UNIT 2

Propositional Logic vs First-Order Predicate Logic

Quantifiers are the whole difference: the ability to say 'for all' instead of listing every case.

	Propositional logic	First-order predicate logic
Basic unit	A whole statement that is true or false	Objects, predicates over them, and relations
Example	RainingToday = True	$\forall x \text{ Human}(x) \rightarrow \text{Mortal}(x)$
Can it generalise?	No - you must state every case separately	Yes - quantifiers state a rule once for all objects
Quantifiers	None	$\forall$ (for all) and $\exists$ (there exists)
Expressive power	Limited - n objects need n separate statements	High - one statement covers an entire domain
Decidable?	Yes - truth tables always terminate	Semi-decidable - proofs may not terminate
Use it for	Small fixed sets of conditions, circuits, simple rules	Domains with many objects and general laws

2.5 REASONING UNDER UNCERTAINTY

UNIT 2

Reasoning Under Uncertainty - Bayes' Theorem

Logic handles what is certainly true. Bayes handles what is probably true.

BAYES' THEOREM

$$P(H | E) = [P(E | H) \cdot P(H)] / P(E)$$

Updates a belief in hypothesis H after observing evidence E. This is how a system reasons when facts are not certain.

THE FOUR TERMS NAMED

$$\text{posterior} = (\text{likelihood} \times \text{prior}) / \text{evidence}$$

P(H) is what you believed before; P(H|E) is what you believe after seeing E.

EXPANDING THE DENOMINATOR

$$P(E) = P(E | H) \cdot P(H) + P(E | \neg H) \cdot P(\neg H)$$

The total probability of the evidence, over both the hypothesis and its negation. Usually how you compute P(E) in practice.

WHY AI NEEDS IT

P(H) the PRIOR - how common the hypothesis is before evidence

P(E|H) the LIKELIHOOD - how expected the evidence is if H holds

P(H|E) the POSTERIOR - the updated belief

The base rate a rare disease stays unlikely even after a positive test, because P(H) is small

Where it appears medical diagnosis, spam filtering, sensor fusion, fault detection

2.5 WORKED EXAMPLE

UNIT 2

Bayes' Theorem - The Medical Test

THE PROBLEM

- A disease affects 1% of the population
- The test is 90% sensitive: $P(\text{positive} \mid \text{disease}) = 0.90$
- The test has a 5% false positive rate: $P(\text{positive} \mid \text{no disease}) = 0.05$
- A patient tests positive. What is the probability they have the disease?

WHY IT MATTERS

- This is the BASE RATE FALLACY - the most common error in probabilistic reasoning
- Most people guess around 90%; the answer is 15%
- It is why screening programmes retest before diagnosing

STEP-BY-STEP SOLUTION

Write down what you know $P(D) = 0.01$, $P(\neg D) = 0.99$, $P(+ \mid D) = 0.90$, $P(+ \mid \neg D) = 0.05$

Step 1 - the numerator $P(+ \mid D) \cdot P(D) = 0.90 \times 0.01 = 0.0090$

Step 2 - the other contribution $P(+ \mid \neg D) \cdot P(\neg D) = 0.05 \times 0.99 = 0.0495$

Step 3 - total probability of a positive test $P(+) = 0.0090 + 0.0495 = 0.0585$

Step 4 - apply Bayes $P(D \mid +) = 0.0090 / 0.0585 = 0.1538$

The answer about 15% - despite a positive result from a 90% sensitive test

Why so low the disease is rare. There are far more healthy people, so 5% of them produces more false positives than 90% of the few sick people produces true positives.

ANSWER $P(\text{disease} \mid \text{positive test}) \approx 0.154$ — the base rate dominates.

CHECK YOUR UNDERSTANDING

Quick Check - Knowledge and Reasoning

UNIT 2

QUICK CHECK

Which representation lets you state 'every student has a roll number' ONCE, rather than repeating it for each student?

- A** Propositional logic
- B** A semantic network with inheritance, or first-order logic with $\forall$
- C** A production rule for each student
- D** A frame with no parent

ANSWER: B Both a semantic network and first-order logic can state a property at the general level and have it apply to every instance - by inheritance down an is-a link, or by the universal quantifier. Propositional logic has no quantifiers, so it needs one separate statement per student.

UNIT II CONSOLIDATION

Activity: Choose the Technique

UNIT 2

GROUP WORK · 12 minutes

In groups of 3-4, take one problem and decide which of this unit's techniques fits - then defend it against the obvious alternative.

1**Pick a problem**

- Campus emergency evacuation routing
- Diagnosing why a laptop will not boot
- Deciding university admission from a rulebook
- Timetabling classes without clashes

2**Decide the approach**

- Search or knowledge representation?
- If search: which algorithm, and what heuristic?
- If knowledge: which representation, and why that one?

3**Justify**

- What property of the problem decided it?
- What would go wrong with the alternative?

4**Anticipate**

- Where does uncertainty enter your problem?
- Would Bayes help, and what would the prior be?

DELIVERABLE One speaker per group: 60 seconds on your technique, the property of the problem that chose it, and what would break if you used the alternative.

UNIT II RECAP

Unit II - The Eight Ideas Worth Remembering

UNIT 2

1**Uninformed search knows the rules but not the direction**

BFS and DFS use only the problem definition. Neither has any estimate of how far the goal is.

2**BFS and DFS differ by one line**

A queue gives breadth-first; a stack gives depth-first. That one choice changes completeness, optimality and memory.

3**BFS is optimal only when step costs are equal**

It minimises the number of edges, not the cost. Weighted graphs need Uniform Cost Search or A*.

4**A heuristic must never overestimate**

Admissibility is the condition for A* to be optimal. An optimistic estimate is safe; an overestimate can lose the best path.

5**A* uses $f(n) = g(n) + h(n)$**

Cost already paid plus estimated cost remaining. With $h = 0$ it becomes Uniform Cost Search.

6**Hill Climbing trades the guarantee for memory**

Constant memory, but it stops at local optima, plateaus and ridges. Random restarts and annealing are the standard fixes.

7**Knowledge representation was the answer to the first winter**

Semantic networks, frames, production systems and logic all encode what an expert knows so the machine need not search for it.

8**Bayes' theorem handles uncertainty**

Posterior = likelihood $\times$ prior / evidence. A positive test for a rare disease still leaves the disease unlikely.

NEXT UP Unit III - Intelligent Agents: agent architecture, the PEAS framework, agent environments, the five agent types, and multi-agent systems.